

3-Mark Questions & Answers

Q1. Differentiate between Data, Database, and DBMS.

Feature	Data	Database	DBMS
Meaning	Raw, unprocessed facts	Organized collection of related data	Software to manage the database
Example	101 , Alice , 90%	Student records table	MySQL, Oracle
Structure	No structure	Structured (tables, files)	System that imposes structure
Usefulness	Meaningless alone	Meaningful when queried	Makes database useful and accessible

Q2. List and briefly explain any three characteristics of a database.

- Data Integrity:** The DBMS enforces rules (constraints) to ensure that all data stored is accurate, valid, and consistent. For example, a student's age cannot be stored as a negative number.
- Data Security:** The DBMS controls who can access which data using authentication (login) and authorization (permissions/privileges). This prevents unauthorized access or modification.
- Reduced Data Redundancy:** In traditional file systems, the same data (e.g., a student's address) might be stored in multiple files, leading to inconsistency. A DBMS minimizes this by centralizing data and using normalization.

Q3. Explain the three levels of Data Abstraction.

The DBMS provides three levels of abstraction to hide complexity from users:

- Physical Level (Internal):** The lowest level. Describes *how* data is physically stored on the disk — file formats, indexing, data blocks, and storage structures. Hidden from all users.
- Logical Level (Conceptual):** The middle level. Describes *what* data is stored and the relationships between them — i.e., the table schemas and constraints. Used by DBAs and programmers.
- View Level (External):** The highest level. Describes only a *part* of the database as seen by a specific user or application. Different users may see different views. Provides simplicity and security.

View Level (What user sees – partial view)
Logical Level (What data exists – full schema)
Physical Level (How data is stored – on disk)

Q4. Differentiate between Logical and Physical Data Independence.

Feature	Logical Data Independence	Physical Data Independence
Definition	Ability to change the logical schema without affecting the view level	Ability to change the physical schema without affecting the logical level
Level of Change	Conceptual/Logical schema changes	Physical/Internal schema changes
Example	Adding a new column to a table without affecting old application code	Changing from sequential file storage to B-tree indexing
Difficulty	Harder to achieve	Easier to achieve

Q5. Explain 1-tier, 2-tier, and 3-tier architecture in brief.

1. **1-Tier Architecture:** The client, application logic, and database all reside on one machine. Used for personal/local applications. Simple but not scalable.
Example: MS Access on a local PC.
2. **2-Tier Architecture:** The client (with application logic) communicates directly with a database server over a network. Better than 1-tier for small teams.
Example: A desktop inventory app connecting to a remote MySQL server.
3. **3-Tier Architecture:** Three distinct layers — (a) **Presentation Tier** (client UI), (b) **Application/Logic Tier** (business rules, middle-ware), and (c) **Database Tier** (data storage). Most scalable and secure; used in modern web applications.
Example: A web browser → Web server (Apache) → Oracle DB.

Q6. What are the tasks/functions offered by a DBMS?

1. **Data Definition:** The DBMS allows users to define the structure (schema) of the database — creating tables, setting data types, and defining constraints using DDL (Data Definition Language).
2. **Data Updation:** The DBMS provides DML (Data Manipulation Language) commands like **INSERT**, **UPDATE**, and **DELETE** to add, modify, or remove data in the database.
3. **Data Retrieval:** Users can query the database to fetch specific data using the **SELECT** statement. The Query Processor optimizes and executes these queries.
4. **User Administration:** The DBA can create user accounts, grant or revoke privileges, and control access to specific tables or operations — ensuring data security.

Q7. Explain any three components of the Storage Manager.

1. **Authorization and Integrity Manager:** Checks that a user has the required permissions before executing a data access operation. Also enforces integrity constraints (e.g., primary key, NOT NULL).
2. **Transaction Manager:** Ensures that database transactions execute reliably. It manages the ACID (Atomicity, Consistency, Isolation, Durability) properties, handles failure recovery, and maintains the database in a consistent state.

3. **Buffer Manager:** Manages the transfer of data between disk storage and main memory (RAM). It fetches data pages into a buffer pool in RAM and decides which pages to write back to disk, minimizing costly disk I/O operations.
-

Q8. Explain DDL interpreter, DML compiler, and Query evaluation engine in brief.

1. **DDL Interpreter:** Interprets Data Definition Language (DDL) statements like `CREATE TABLE` and `DROP TABLE`. It records the definitions (metadata) in the **data dictionary**.
 2. **DML Compiler:** Translates DML queries (like `SELECT`, `UPDATE`) written in high-level SQL into low-level query plans (procedural instructions) that the query evaluation engine can execute.
 3. **Query Evaluation Engine:** Executes the low-level query plan generated by the DML compiler. It coordinates with the Storage Manager to physically access the data on disk and return the result to the user.
-

Q9. List any three responsibilities of a DBA.

1. **Schema Definition:** The DBA writes DDL statements to create the original database schema (tables, views, indexes, constraints) and loads the initial data.
 2. **Granting User Access:** The DBA creates user accounts and uses `GRANT` and `REVOKE` commands to control which users have access to which parts of the database, ensuring security.
 3. **Routine Maintenance & Monitoring:** The DBA monitors system performance, performs periodic backups to prevent data loss, plans recovery procedures for failures, and tunes the database configuration for optimal performance.
-

Q10. Differentiate between Application Programmers and Sophisticated Users.

Feature	Application Programmers	Sophisticated Users
Who are they?	IT professionals who develop database applications	Engineers, scientists, analysts
How they interact	Write programs using DML calls embedded in host languages (Java, Python)	Write SQL queries directly or use tools like Power BI
Output	Pre-built applications used by naive users	Ad-hoc queries and data analysis reports
Example	A developer who builds a library management app	A business analyst writing complex JOIN queries

Q11. Explain the need for a database (any three points).

1. **Eliminating Data Redundancy and Inconsistency:** In file systems, the same data (e.g., a customer's phone number) is stored in multiple files, leading to inconsistency when one is updated but others are not. A DBMS stores data centrally, eliminating redundancy.
2. **Difficulty in Accessing Data:** Retrieving data from flat files requires writing custom programs for every query. A DBMS provides a powerful query language (SQL) that allows easy, ad-hoc retrieval of data without programming.

3. **Data Security and Integrity:** File systems have limited access control. A DBMS provides fine-grained security using user accounts, privileges, and integrity constraints (like NOT NULL, FOREIGN KEY), ensuring only valid and authorized data operations occur.

Q12. Write short notes on Specialized Users.

Specialized Users are sophisticated users who write specialized database applications that don't fit the traditional data processing framework. They work with advanced database features and technologies such as:

- **Knowledge Base Systems:** Systems that store rules and facts (AI/Expert systems).
- **CAD Systems:** Computer-Aided Design systems that store complex geometric and engineering data.
- **Geographic Information Systems (GIS):** Applications that store and query spatial/map data.

These users often interact with specialized DBMS types (e.g., Graph DBMS, Spatial DBMS) that go beyond the standard relational model.

Q13. Differentiate File System and Database approach (any 3 points).

Feature	File System	Database System
Data Redundancy	High — same data stored in multiple files, leading to inconsistency	Low — data stored centrally, redundancy minimized via normalization
Data Access	Requires custom programs to access/query data; no standard query language	Standard SQL provides easy, powerful, and flexible data retrieval
Data Security	Limited security; access is mainly at the file/folder level via OS	Fine-grained security with user accounts, roles, and privileges per table/column
Data Integrity	No built-in constraints; integrity is managed manually in programs	DBMS enforces constraints (PK, FK, NOT NULL) automatically
Concurrency	Very limited support; simultaneous file access can corrupt data	Built-in concurrency control ensures safe simultaneous multi-user access

Q14. Explain Entity, Entity Set, and Attributes with an example.

- **Entity:** A specific, distinguishable object or concept in the real world. Example: Student "Suraj" with ID 101.
- **Entity Set:** A collection of similar entities sharing the same structure. Example: STUDENT entity set containing all student entities.
- **Attributes:** Properties or characteristics describing each entity in an entity set. Example: Roll_No, Name, Branch, and GPA describe each STUDENT entity.

Q15. Explain the components of a Relationship Set with an example.

A **Relationship Set** is a mathematical relation among ≥ 2 entity sets.

- * **Participating Entity Sets:** The entity sets linked by the relationship.
- * **Relationship Instances:** Individual associations between specific entities.
- * **Descriptive Attributes:** Attributes that belong to the relationship set itself (rather than to participating entity sets).

* **Example:** WORKS_ON connects EMPLOYEE and PROJECT. It has a descriptive attribute Hours_Worked which records how many hours a specific employee worked on a specific project.

Q16. Differentiate between Simple and Composite attributes with examples.

Feature	Simple Attribute	Composite Attribute
Definition	Atomic; cannot be divided into smaller sub-components	Can be divided into smaller sub-components with independent meanings
Structure	Single component	Multi-component hierarchy
Example	Age, Salary, Roll_No	Name (First_Name, Last_Name), Address (Street, City, Zip)
Query Use	Queried directly	Sub-parts can be queried individually (e.g., filter by City)

Q17. Differentiate between Single-valued and Multivalued attributes.

Feature	Single-Valued Attribute	Multivalued Attribute
Values per Entity	Holds exactly one value per entity instance	Can hold zero, one, or multiple values per entity instance
ER Symbol	Single Oval / Ellipse	Double Oval / Ellipse
Example	Date_of_Birth, SSN	Phone_Number (work, home, mobile), Degrees
Relational Mapping	Map directly as a column in the main entity table	Requires a separate table with primary key + attribute

Q18. Explain Derived Attributes with an example.

A **Derived Attribute** is an attribute whose value is not physically stored in the database but is computed dynamically on demand from other stored attributes or system functions.

* **ER Symbol:** Dashed Ellipse.

* **Example:** Age is derived from Current_Date - Date_of_Birth. Storing Date_of_Birth is sufficient; computing Age on the fly ensures it is always accurate without needing daily database updates.

Q19. Explain the ER diagram notation for entity sets, attributes, and relationships (symbols used).

Rectangle Double Rectangle Ellipse / Oval

+-----+ +=====+ /-----\

| Strong Entity | || Weak Entity || (Attribute)

+-----+ +=====+ -----/

Double Ellipse Dashed Ellipse Diamond

/===== \ / - - - - \ \

((Multivalued)) (Derived Attribute) / \ Relationship /

\===== / \ - - - - - / \ / Set /
V
,

Q20. Explain the four types of mapping cardinality with examples.

1. **One-to-One (1:1):** An entity in A is associated with at most one entity in B, and vice versa. Example: EMPLOYEE *manages* DEPARTMENT (each employee manages at most 1 department; each department is managed by 1 employee).
2. **One-to-Many (1:N):** An entity in A is associated with any number of entities in B, but an entity in B is associated with at most one entity in A. Example: DEPARTMENT *employs* EMPLOYEE.
3. **Many-to-One (N:1):** Multiple entities in A associated with at most one entity in B. Example: STUDENT *belongs to* DEPARTMENT.
4. **Many-to-Many (M:N):** An entity in A is associated with any number of entities in B, and vice versa. Example: STUDENT *enrolls in* COURSE.

Q21. Differentiate between Total and Partial Participation with examples.

Feature	Total Participation	Partial Participation
Constraint	Every entity in the set must participate in the relationship	Some entities may not participate in the relationship
Dependency	Existence dependency (mandatory)	Optional participation
ER Line Symbol	Double Line	Single Line
Example	Every LOAN <i>must belong to</i> a CUSTOMER	Not every EMPLOYEE <i>manages</i> a DEPARTMENT

Q22. Explain weak entity set and strong entity set with an example.

- **Strong Entity Set:** Has a primary key that uniquely identifies every entity. Does not depend on any other entity set. Example: EMPLOYEE(Emp_ID, Name, Salary).
- **Weak Entity Set:** Lacks a primary key of its own. Entities are identified by combining the owner's primary key + its own discriminator (partial key). It has total participation in an identifying relationship.
- **Example:** DEPENDENT(Dep_Name, Relationship) linked to EMPLOYEE via identifying relationship HAS_DEPENDENT. Primary Key of DEPENDENT = (Emp_ID, Dep_Name).

Q23. Explain how a primary key is derived for a relationship set (1:1, 1:N, M:N).

The primary key of a relationship set depends on its mapping cardinality:

* **Many-to-Many (M:N):** Primary key = **Union of Primary Keys** of both participating entity sets. PK = (Emp_ID, Proj_ID).

* **One-to-Many (1:N) / Many-to-One (N:1):** Primary key = **Primary Key of the "Many" side** entity set. PK = (Emp_ID).

* **One-to-One (1:1):** Primary key = **Primary Key of ANY ONE** of the participating entity sets. PK = (Emp_ID) OR (Dept_ID).

Q24. Explain Role in a relationship with an example (e.g., prerequisite).

A **Role** is the explicit label given to an entity set's participation in a relationship set when the same entity set participates **more than once** (recursive relationship).

* **Example:** COURSE has a recursive relationship PREREQUISITE.

* Role 1: Main_Course (the course being taken).

* Role 2: Prereq_Course (the course required before taking the main course).

* Explicit role labels on ER lines clarify which course depends on which.

Q25. Explain Overlapping and Disjoint Specialization with examples.

- **Disjoint Specialization:** An entity can belong to **at most one** lower-level subclass. Represented by 'd' in ER circle. Example: EMPLOYEE is either HOURLY_EMPLOYEE OR SALARIED_EMPLOYEE (cannot be both).
 - **Overlapping Specialization:** An entity can belong to **multiple** lower-level subclasses simultaneously. Represented by 'o' in ER circle. Example: A person at a university can be both a STUDENT and an EMPLOYEE (e.g., Teaching Assistant).
-

Q26. Explain the Completeness Constraint (Total/Partial generalization).

The **Completeness Constraint** specifies whether every entity in the superclass **MUST** belong to at least one subclass:

* **Total Generalization/Specialization:** Every higher-level entity must belong to a lower-level subclass. (Double line connecting superclass to circle). Example: Every PATIENT must be either IN_PATIENT or OUT_PATIENT.

* **Partial Generalization/Specialization:** Some higher-level entities may not belong to any subclass. (Single line). Example: An EMPLOYEE might not be a SECRETARY, ENGINEER, or TECHNICIAN (could be a Manager).

Q27. Explain Attribute Inheritance in generalization/specialization.

Attribute Inheritance is a fundamental property of ISA hierarchies.

* Every entity in a lower-level subclass **automatically inherits all attributes** of its higher-level superclass.

* It also inherits all **relationships** in which the superclass participates.

* **Example:** EMPLOYEE has attributes Emp_ID, Name, Salary. Subclass SECRETARY inherits Emp_ID, Name, Salary and adds its own specific attribute Typing_Speed.

Q28. Explain Condition-defined vs User-defined lower-level entity sets.

- **Condition-Defined (Predicate-Defined):** Membership in a subclass is automatically determined by an explicit condition/predicate on an attribute of the superclass. Example: If Job_Type = 'Secretary', entity automatically belongs to SECRETARY subclass.
 - **User-Defined:** Membership in a subclass is manually assigned by a database user/administrator when no explicit condition exists. Example: Assigning specific employees to a special RESEARCH_PROJECT_TEAM subclass.
-

Q29. What is a redundant attribute? Explain with the Student-Department example.

A **Redundant Attribute** is an attribute that introduces duplicate information that can already be derived from existing relationships or other tables.

* **Example:** In the STUDENT entity set storing (Roll_No, Name, Dept_ID, Dept_Name), the attribute Dept_Name is **redundant** because Dept_ID is already stored in STUDENT, and the DEPARTMENT entity set maps Dept_ID → Dept_Name.

* **Solution:** Remove Dept_Name from STUDENT to maintain normalization and avoid inconsistency.

Q30. Explain Key Constraint on Entity Set and Relationship Set.

- **Key Constraint on Entity Set:** Ensures every entity has a unique identifier (Primary Key / Candidate Key) so no two entities share identical key values.
 - **Key Constraint on Relationship Set:** Restricts how many relationship instances an entity can participate in. In a binary relationship R between A and B, a key constraint on A means each entity in A maps to **at most one** entity in B (enforcing 1:1 or 1:N cardinality).
-

Q31. Identify the entity sets and relationships for the Hospital Management System, and classify each relationship as 1:1, 1:N, or M:N.

- **Entity Sets:** PATIENT, DOCTOR, ROOM, MEDICINE.
 - **Relationships & Cardinalities:**
 1. DOCTOR *treats* PATIENT **(1:N or M:N)**: One doctor treats multiple patients; a patient can be treated by multiple doctors (M:N).
 2. PATIENT *assigned* ROOM **(N:1)**: Multiple patients can occupy a room over time, but a patient is in at most 1 room at a time.
 3. DOCTOR *prescribes* MEDICINE **(M:N)**: Doctors prescribe multiple medicines; medicines are prescribed by multiple doctors.
-

Q32. Explain the Foreign Key approach for mapping a binary 1:1 relationship, using EMPLOYEE-DEPARTMENT (Manages).

To map a binary 1:1 relationship MANAGES between EMPLOYEE and DEPARTMENT:

1. Create relation EMPLOYEE(Emp_ID, Name, Salary).
2. Create relation DEPARTMENT(Dept_ID, Dept_Name).
3. Choose the entity set with **Total Participation** (or either side if both are partial/total) to store the Foreign Key. Since every department has a manager:

* Add Manager_Emp_ID as a **Foreign Key** in DEPARTMENT referencing Emp_ID of EMPLOYEE.

* Add any descriptive attributes of MANAGES (e.g., Start_Date) into DEPARTMENT.

Q33. For the Online Shopping System, describe the mapping of "Customer places Order" (which side gets the foreign key, and why).

- **Relationship:** CUSTOMER *places* ORDER is a **1:N relationship** (1 Customer places N Orders; 1 Order belongs to 1 Customer).
- **Mapping Rule:** The Foreign Key is **always placed on the "N" (Many) side** (ORDER table).
- **Why:** Each Order belongs to exactly one Customer. Placing Customer_ID as a Foreign Key in ORDER allows easy referencing without NULL values or duplicate entries. Placing it on CUSTOMER would require storing a multivalued

array of Order IDs, violating 1NF.

Q34. Explain how multivalued attributes are mapped to a relational schema, with an example (e.g., DEPT_LOCATIONS).

- **Mapping Rule:** A multivalued attribute cannot be stored directly in a relational table (violates 1NF). Create a **new separate relation** containing:
 1. The multivalued attribute itself.
 2. The Primary Key of the parent entity set as a **Foreign Key**.
- **Primary Key of new relation:** Combination of (Parent Primary Key + Multivalued Attribute).
- **Example (DEPARTMENT with multivalued Locations):**
 - DEPARTMENT(Dept_ID, Dept_Name)
 - DEPT_LOCATIONS(Dept_ID, Location) where PK = (Dept_ID, Location) and Dept_ID is FK.

Q35. Differentiate "Multiple relations – Superclass and subclasses" vs "Single relation with one type attribute" for specialization mapping.

Feature	Option 1: Multiple Relations (Super + Sub)	Option 3: Single Relation (One Type Attr)
Tables Created	1 table for superclass + 1 table per subclass	Exactly 1 single unified table
Handling Subclass Attributes	Subclass tables store only specific attributes	Single table holds ALL attributes for ALL subclasses
NULL Values	Zero NULL values	Many NULL values for non-applicable subclass attributes
Best Used When	Subclasses have many specific attributes	Specialization is disjoint with few specific attributes

Q36. For the Movie Ticket Booking System, identify entity sets and the cardinality of each relationship.

- **Entity Sets:** USER, THEATRE, SHOW, TICKET.
- **Relationships & Cardinalities:**
 1. USER *books* TICKET (**1:N**): 1 user can book multiple tickets; 1 ticket belongs to 1 user.
 2. THEATRE *screens* SHOW (**1:N**): 1 theatre screens multiple shows; 1 show belongs to 1 theatre.
 3. SHOW *issues* TICKET (**1:N**): 1 show issues multiple tickets; 1 ticket is for 1 show.

Q37. Explain why an M:N relationship set requires a new relation, unlike 1:N relationships.

- **In 1:N relationships:** The "N" side entity participates in at most 1 relationship instance. Hence, we can cleanly place the "1" side's primary key as a Foreign Key directly inside the "N" side table without violating 1NF.
- **In M:N relationships:** An entity in A connects to multiple entities in B, and vice versa.
- If we tried to put Foreign Keys inside A or B, we would need to store multiple values per cell (violating 1NF) or duplicate rows (violating Primary Key uniqueness).

- **Solution:** An M:N relationship requires a **separate junction table** containing (PK_A, PK_B) as Foreign Keys.
-

Q38. What are the three approaches for mapping a binary 1:1 relationship? When is each appropriate?

1. **Foreign Key Approach:** Add Foreign Key of one relation into the other. *Best when one entity set has total participation (place FK in total side).*
 2. **Merged Relation Approach:** Merge both entity sets into a single unified relation. *Best when BOTH entity sets have total participation in 1:1.*
 3. **Cross-Reference / Junction Table Approach:** Create a separate relationship table (PK_A, PK_B). *Best when BOTH sides have partial participation to prevent NULL values.*
-